Spring Boot Database CRUD

with Thymeleaf, Spring Data JPA, and MySQL

1. From REST Data to Persistent Database Applications

A Spring Boot application often needs more than temporary or hard-coded data. In a database-backed application, data must remain available after the application stops and restarts. This requires a **persistence layer** that connects the application to a database.

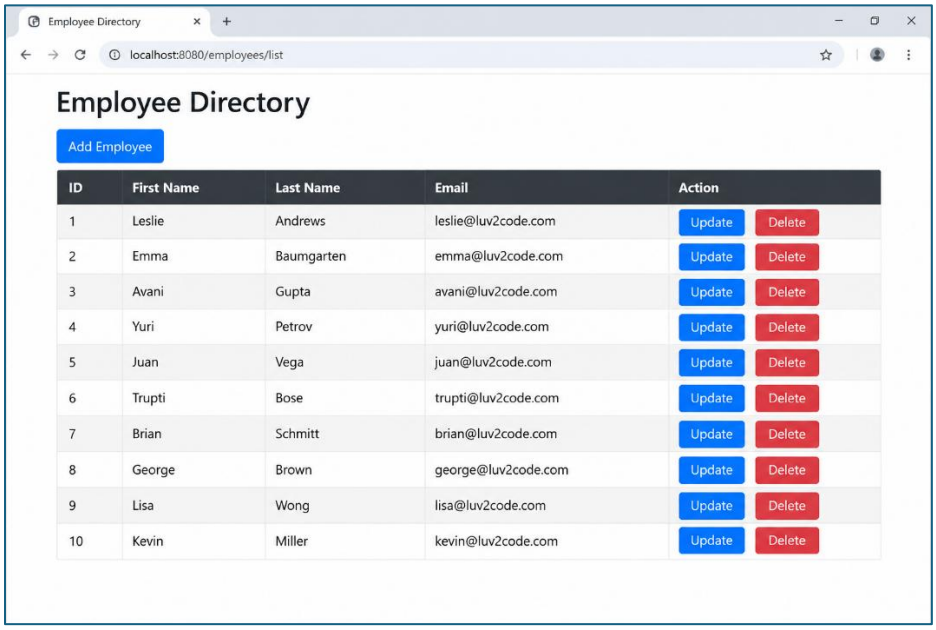
In this lesson, we build a complete **CRUD web application** for managing employees. CRUD represents the four fundamental data-management operations:

Operation	Meaning	Employee Example
Create	Add new data	Add an employee
Read	Retrieve existing data	Display employees
Update	Modify existing data	Edit employee information
Delete	Remove data	Delete an employee

The application uses **Spring Boot**, **Spring Data JPA**, **MySQL**, **Thymeleaf**, and a layered architecture. The final interface allows users to list, add, update, and delete employee records.

Learning Goal

Understand how data travels from a browser through the Spring MVC application layers to a relational database—and back to a Thymeleaf-generated web page.

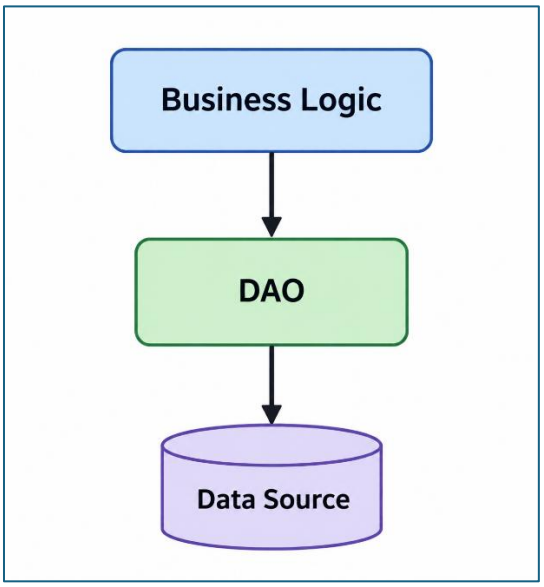


2. Understanding the DAO Pattern

2.1 What Is a DAO?

DAO stands for **Data Access Object**. It is a software design pattern that isolates the code responsible for interacting with a data source.

Instead of allowing controllers or business logic to communicate directly with a database, database operations are placed behind a dedicated data-access component.



The DAO layer commonly provides operations such as:

`save()`

`findById()`

`findAll()`

`deleteById()`

This creates **separation of concerns**: application logic does not need to know the low-level details of how information is retrieved or stored. This improves maintainability and makes it easier to change the data-access mechanism.

---

3. DAO with Spring Data JPA

Spring Boot simplifies the DAO pattern significantly through **Spring Data JPA**.

Instead of manually implementing common CRUD operations, we can create a repository interface:

```
public interface EmployeeRepository
        0 references
        extends JpaRepository<Employee, Integer> {
}
```

`JpaRepository` already provides implementations for common persistence operations. Spring creates the repository implementation automatically at runtime.

For example, the repository can support operations conceptually equivalent to:

```
employeeRepository.findAll();
employeeRepository.findById(id);
employeeRepository.save(employee);
employeeRepository.deleteById(id);
```

Tip — Repository as DAO

In this architecture, **`EmployeeRepository`** effectively serves as the DAO/data-access layer. We therefore do not need to manually write SQL for every basic CRUD operation.

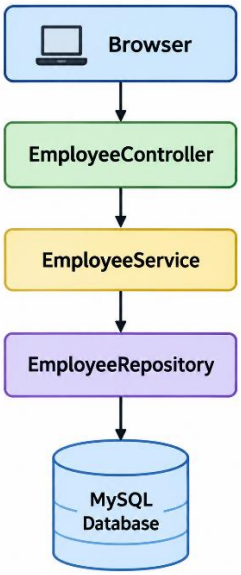
For specialized data-access requirements, a custom DAO can instead be implemented using mechanisms such as **`JdbcTemplate`** or **`EntityManager`**.

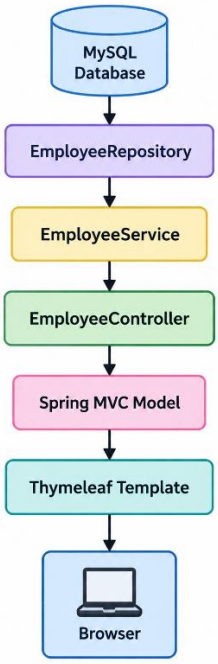
---

4. Application Architecture

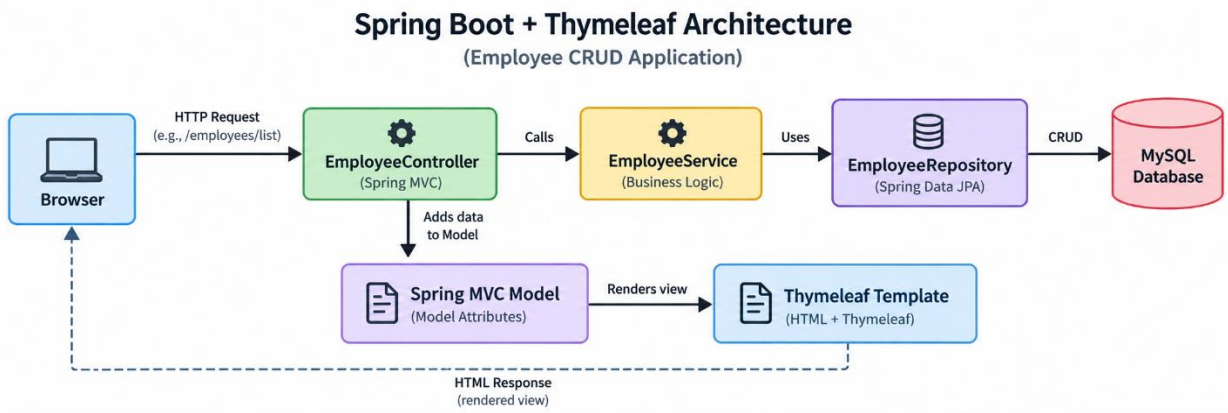
The employee application follows a layered architecture.

For displaying information, the data returns through the application and is placed in the Spring MVC Model. Thymeleaf then renders the HTML response. See the following figure.





The controller handles browser requests, the service provides the application/service layer, the repository performs persistence operations, and Thymeleaf generates the resulting view.



---

5. Preparing the MySQL Database

Before connecting Spring Boot to MySQL, the database environment must be prepared.

Step 1 — Open MySQL Workbench

Start **MySQL Workbench** and connect using the database administrator account.

Step 2 — Create the Application Database User

Execute the SQL script that creates the application user.

The configuration uses:

`Username: springstudent`

`Password: springstudent`

After execution, verify the account under **Users and Privileges**.

Step 3 — Create the Database Schema and Table

Run the database initialization SQL script.

The setup creates a schema and a table containing fields such as:

- **`id`**
- **`first_name`**
- **`last_name`**
- **`email`**

The database can then be inspected from MySQL Workbench to verify that the schema and table were created correctly.

Note
At this stage, verifying the database independently of Spring Boot is useful. If the table cannot be queried successfully in MySQL Workbench, connecting the Spring application will not solve the underlying database problem.

---

6. Preparing the Spring Boot Project

The project requires several Spring Boot components.

The main dependencies include:

- **`Spring Web`**
- **`Spring Data JPA`**
- **`Thymeleaf`**
- **`MySQL Driver`**

The Maven configuration must include database support in addition to Spring Data JPA, Thymeleaf, and Spring Web. After changing pom.xml, reload the Maven project so the dependencies are downloaded and added to the classpath.

A practical project structure is:

```
src/main/java
└── com.example.application
    ├── controller
    │   └── EmployeeController.java
    ├── entity
    │   └── Employee.java
    ├── repository
    │   └── EmployeeRepository.java
    ├── service
    │   ├── EmployeeService.java
    │   └── EmployeeServiceImpl.java
    └── Application.java

src/main/resources
├── static
│   └── index.html
├── templates
│   └── employees
│       ├── list-employees.html
│       └── employee-form.html
└── application.properties
```

The separation of controller, service, repository, and entity packages reflects the application's layered architecture.

---

7. Reading Employees from the Database

The first CRUD operation to implement is **Read**.

Previously, employee objects may have been hard-coded inside the controller. Once database integration is introduced, those hard-coded objects should be removed. The employee information will instead come from the database.

Step 1 — Inject the Employee Service

The controller needs access to EmployeeService.

Constructor injection can be used:

```java
private EmployeeService employeeService;

public EmployeeController(EmployeeService employeeService) {
this.employeeService = employeeService;
}
```

When a Spring-managed class contains only one constructor, explicitly adding @Autowired to that constructor is optional.

Step 2 — Retrieve the Employees

The controller retrieves employees through the service:

```
List<Employee> employees = employeeService.findAll();
```

Step 3 — Add Employees to the Model

The result is placed into the Spring MVC model:

```
theModel.addAttribute("employees", employees);
```

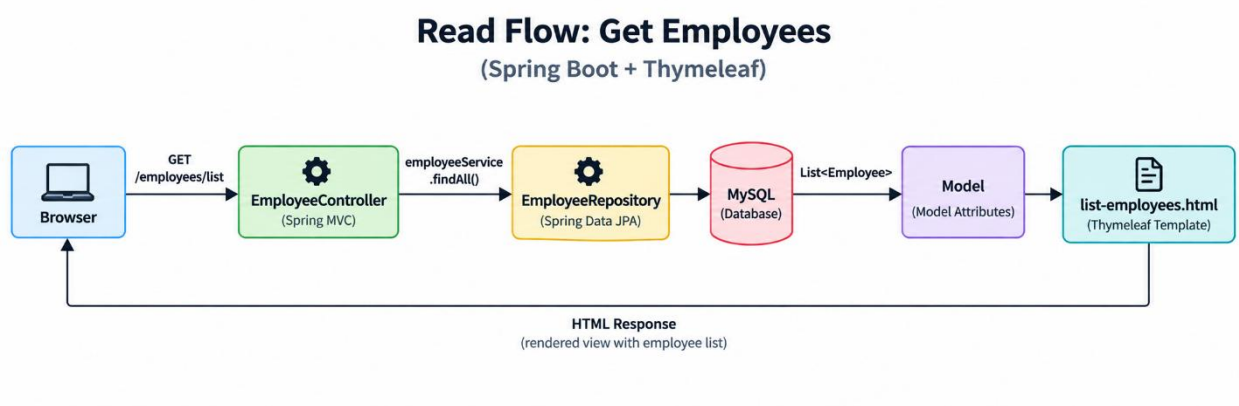
The model becomes the bridge between the controller and the Thymeleaf template.

Step 4 — Return the Thymeleaf View

The controller returns:

```
return "employees/list-employees";
```

The complete flow is therefore:



The controller retrieves database data through the service/repository chain and places it into the model for Thymeleaf rendering.

---

8. Redirecting the Root URL

Without additional configuration, visiting:

`http://localhost:8080`

may result in a **404** because the employee list is mapped elsewhere.

One simple solution is to create:

`src/main/resources/static/index.html`

and redirect the user to:

`/employees/list`

using an HTML refresh:

```
<meta http-equiv="refresh"
      content="0; URL='employees/list'">
```

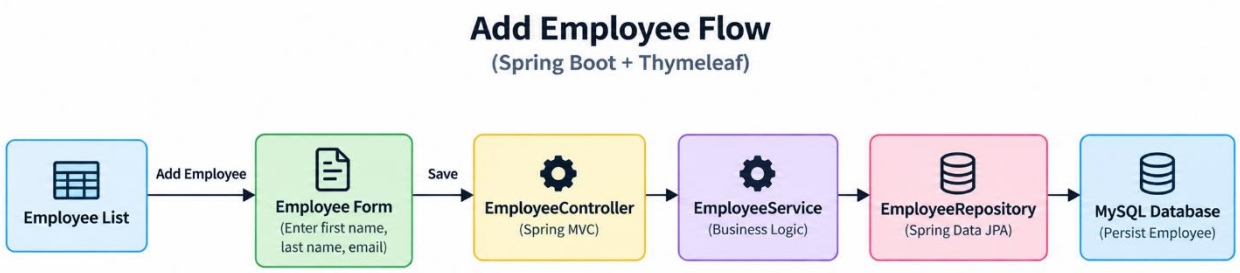
After restarting the application, accessing the root URL redirects to the employee directory.

---

9. Creating an Employee

The next CRUD operation is **Create**.

The workflow is:



The UI introduces an **Add Employee** button that opens a form containing first name, last name, and email fields.

---

10. Step-by-Step: Add Employee Form

Step 1 — Add the Button

In **`list-employees.html`**, create a Thymeleaf link pointing to:

`/employees/showFormForAdd`

For example:

```
<a th:href="@{/employees/showFormForAdd}">
Add Employee
</a>
```

Bootstrap classes can be added for presentation.

Step 2 — Create the Controller Mapping

The controller creates an empty Employee object:

```
@GetMapping("/showFormForAdd")
public String showFormForAdd(Model theModel) {

Employee employee = new Employee();

theModel.addAttribute("employee", employee);

return "employees/employee-form";
}
```

The important line is:

`theModel.addAttribute("employee", employee);`

It exposes the object to Thymeleaf under the name employee, allowing the form fields to bind to its properties.

---

11. Thymeleaf Form Data Binding

Thymeleaf provides special attributes for connecting HTML forms to Spring MVC objects.

Three important attributes are:

Attribute	Purpose
`th:action`	Defines where the form is submitted
`th:object`	Identifies the model object used by the form
`th:field`	Binds an input to a property of that object

For example:

```
<form th:action="@{/employees/save}"
th:object="${employee}"
method="POST">

<input type="text"
        th:field="*{firstName}"
        placeholder="First name">

<input type="text"
        th:field="*{lastName}"
        placeholder="Last name">

<input type="text"
        th:field="*{email}"
        placeholder="Email">

<button type="submit">Save</button>

</form>
```

Here:

`th:object="${employee}"`

connects the entire form to the employee model attribute.

Then:

`th:field="*{firstName}"`

binds that input to:

`employee.firstName`

Thymeleaf and Spring MVC use the object's getters and setters during this binding process.

Tip
Think of **`th:object`** as selecting the Java object and **`th:field`** as selecting one property of that object.

---

12. Saving the Employee

When the user clicks **Save**, the form submits a POST request.

The controller processes it using:

```java
@PostMapping("/save")
public String saveEmployee(
        @ModelAttribute("employee") Employee employee) {

employeeService.save(employee);

return "redirect:/employees/list";
}
```

`@ModelAttribute` reconstructs/populates the **`Employee`** object using the submitted form values.

Then:

`employeeService.save(employee);`

passes the object through the service and repository layers to the database.

---

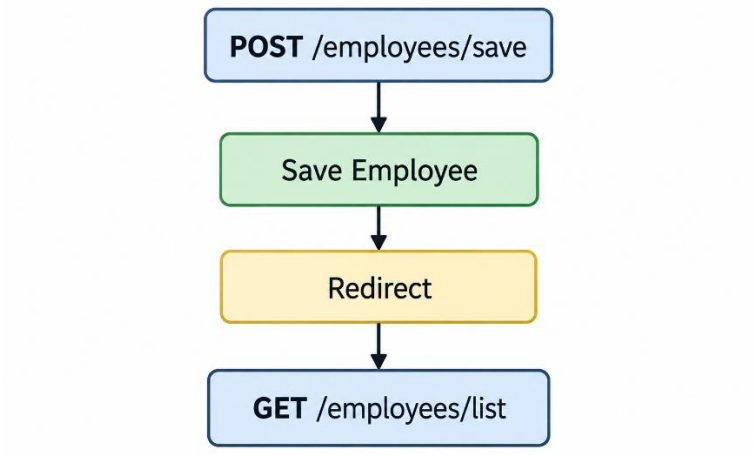
13. Post/Redirect/Get Pattern

Notice that the save method does **not** directly return the employee-list template.

Instead:

`return "redirect:/employees/list";`

This implements the **Post/Redirect/Get (PRG)** pattern:



This avoids accidental duplicate form submissions when the user refreshes the page after saving.

Pitfall — Returning the View Directly
Returning the view immediately after a successful POST can leave the browser associated with the original POST request. A refresh may therefore attempt to submit the form again.

---

14. Sorting Employees

Spring Data JPA can also derive repository queries from appropriately structured method names.

The employee repository can define a method that retrieves employees ordered by last name:

```
List<Employee> findAllByOrderByLastNameAsc();
```

The service's `findAll()` implementation can then call this method instead of the standard repository `findAll()`.

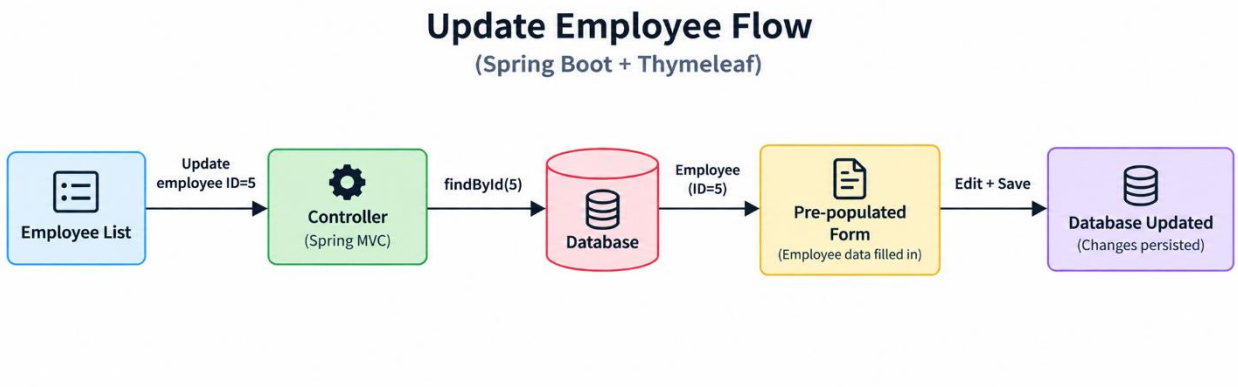
The resulting employee directory displays records alphabetically by last name.

---

15. Updating an Employee

The **Update** operation introduces an important requirement: the application must know **which existing database record** is being edited.

The process is:



Each employee row contains an Update link with that employee's ID embedded in the request.

---

16. Step-by-Step: Update an Employee

Step 1 — Add an Action Column

Add a new table heading:

`<th>Action</th>`

Each employee row receives an **Update** link.

Conceptually, the generated URL looks like:

`/employees/showFormForUpdate?employeeId=5`

The ID allows the backend to determine which employee should be loaded.

Step 2 — Receive the Employee ID

The controller receives the ID as a request parameter:

```
@GetMapping("/showFormForUpdate")
public String showFormForUpdate(
        @RequestParam("employeeId") int id,
        Model theModel) {

Employee employee = employeeService.findById(id);

theModel.addAttribute("employee", employee);

return "employees/employee-form";
}
```

The key operation is:

`employeeService.findById(id);`

The application retrieves the corresponding employee from the database and adds it to the model.

Step 3 — Pre-Populate the Form

Because the model now contains an existing **`Employee`**, the same Thymeleaf form can display its existing property values.

For example:

`th:field="*{firstName}"`

is initialized using the current employee information.

17. Why the Employee ID Must Be Preserved

The update form must also preserve the employee's ID:

```
<input type="hidden" th:field="*{id}">
```

The user does not need to edit or even see this value, but the application needs it to identify the database record being updated.

Without the ID, the submitted object could no longer reliably identify the existing employee.

The hidden field binds the employee ID to the model object and allows the application to determine which employee should be updated.

Important
A hidden field is still submitted with the form even though it is not visible to the user.

---

18. Reusing the Save Operation for Create and Update

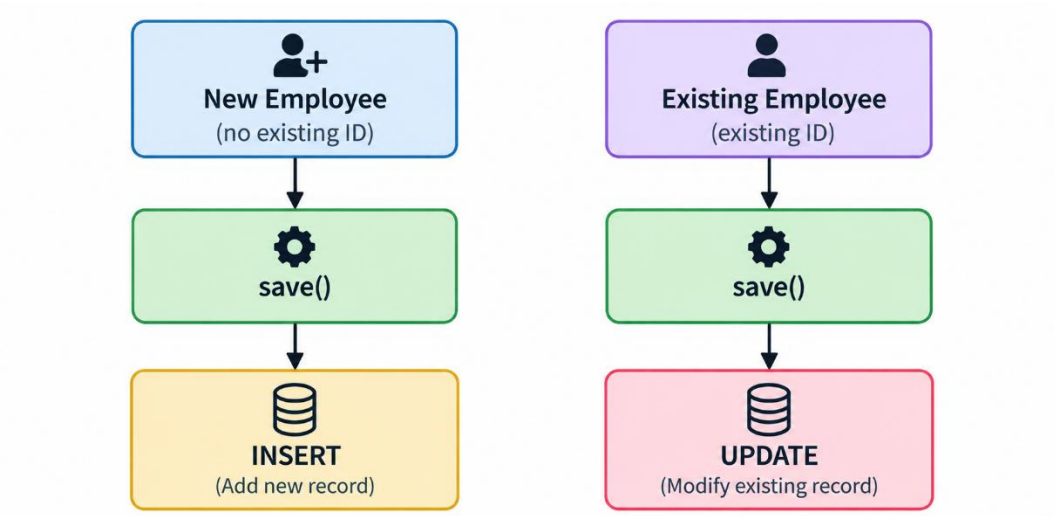
An important design advantage is that the existing save workflow can be reused.

For both adding and updating:

```
employeeService.save(employee);
```

is called.

Conceptually:



The same save functionality therefore supports both creation and modification of employees, provided the existing employee's ID is preserved during the update process.

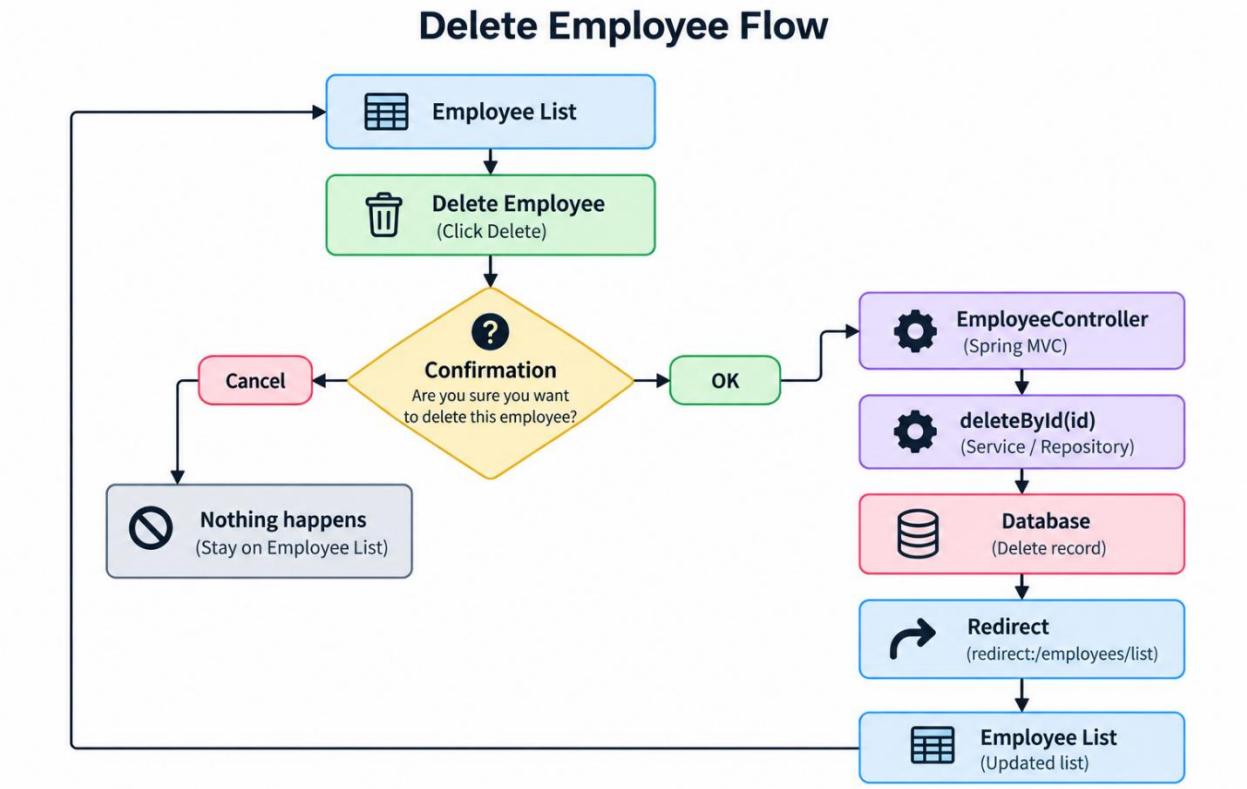
---

19. Deleting an Employee

The final CRUD operation is **Delete**.

Each employee receives a Delete button. The employee ID is embedded in the link so the backend knows which record should be removed. A JavaScript confirmation prompt allows the user to cancel the operation before deletion occurs.

The workflow is:



---

20. Step-by-Step: Delete an Employee

Step 1 — Add the Delete Link

The employee ID must be included in the request.

Conceptually:

`/employees/delete?employeeId=5`

The Thymeleaf link associates each Delete button with the corresponding employee.

Step 2 — Ask for Confirmation

A JavaScript confirmation can prevent accidental deletion:

```
onclick="if (!(confirm('Are you sure you want to delete this employee?'))) return false"
```

If the user selects **Cancel**, the request is not sent.

If the user selects **OK**, the deletion request continues.

Step 3 — Handle the Request

The controller receives the ID:

```
@GetMapping("/delete")
public String delete(
        @RequestParam("employeeId") int id) {

employeeService.deleteById(id);

return "redirect:/employees/list";
}
```

The essential operation is:

`employeeService.deleteById(id);`

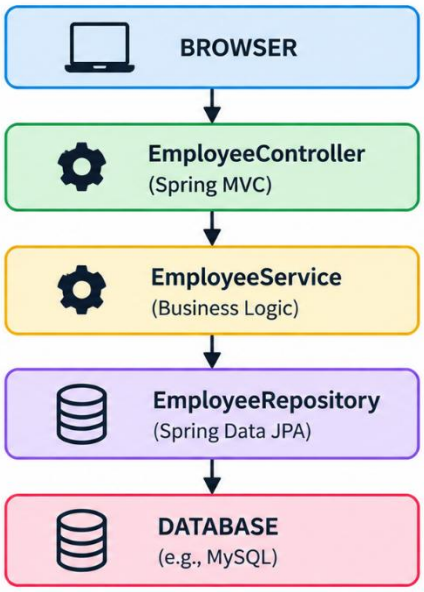
After deletion, the browser is redirected to the employee list so the updated database contents are displayed.

Warning
Delete operations permanently modify database data. A confirmation prompt is a useful UI safeguard against accidental clicks.

---

21. Complete CRUD Request Flow

At this point, the application implements all four CRUD operations:



The application's major workflows are:

User Action	Controller Responsibility	Database Operation
List Employees	Retrieve employees	**READ**
Add Employee	Receive new employee	**CREATE**
Update Employee	Retrieve and save existing employee	**READ + UPDATE**
Delete Employee	Receive employee ID and remove record	**DELETE**

The development sequence—list, add, update, and delete—completes the application's full CRUD functionality.

---

22. Practical Lab: Testing the Complete Application

Run the application and systematically verify the complete workflow.

Test 1 — Read

Navigate to:

`http://localhost:8080/employees/list`

Verify that employee records displayed in the browser correspond to records stored in MySQL.

Test 2 — Create

Click **Add Employee**, enter a new employee, and save.

Verify:

Form

→ **Save**

→ **Database**

→ **Redirect**

→ **Employee appears in list**

Also query the database directly in MySQL Workbench.

Test 3 — Update

Select an existing employee and click **Update**.

Verify that:

Correct employee is retrieved

→ **Form is pre-populated**

→ **ID is preserved**

→ **Changes are saved**

→ **Updated information appears**

Test 4 — Delete

Select an employee and click **Delete**.

Verify both cases:

Cancel → employee remains

OK → employee is removed

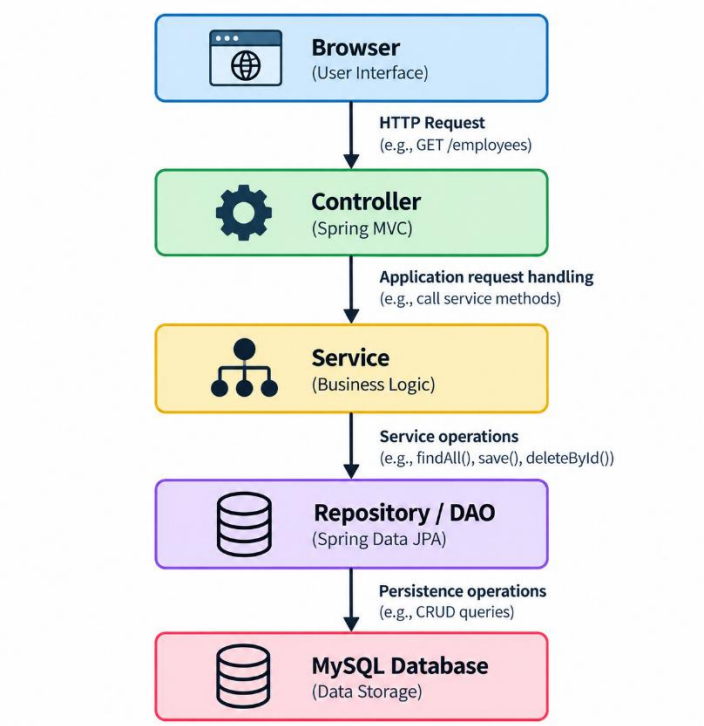
Then verify the deletion directly in MySQL Workbench.

Practical Tip
Do not test only the browser interface. Querying the MySQL table after Create, Update, and Delete operations demonstrates that the application is modifying persistent data rather than merely changing what appears on the page.

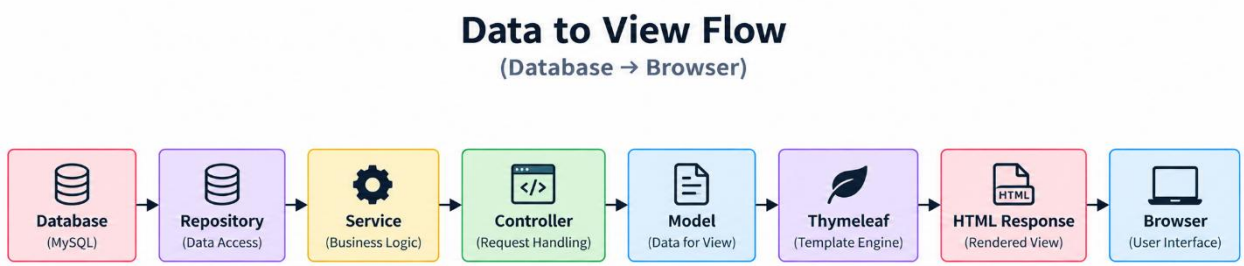
---

23. Final Architecture Perspective

The most important concept in this project is not any individual CRUD method. It is the **separation of responsibilities across layers**:



For rendered web responses:



This architecture keeps HTTP request handling, application/service logic, persistence, and presentation responsibilities separated. The result is a Spring Boot application in which **Thymeleaf provides the Web UI, Spring MVC coordinates requests and models, Spring Data JPA provides repository-based persistence, and MySQL stores the application data**.